Université Paris Nanterre

Université Paris Nanterre

Master 1 MIAGE

Combinatorial Optimization

GraphBench Challenge

Automatic Refutation of Graph Theory Conjectures

Authors:	Seyed Amineddin MIRI and Marie BIBI
Course:	Combinatorial Optimization
Instructor:	François DELBOT
Academic year:	2025–2026
Submission date:	May 12, 2026
Repository:	https://github.com/Aminmiri82/projet-optimisation-combinatoire

Refuted conjectures:	100/100
Final score ($\sum_i c_i$):	25.28
Part 1 total time:	25.28 s
Part 1 average time:	0.253 s
FunSearch (GPT-5.5, `candidate_006`):	100/100, 126.54 s
FunSearch (DeepSeek, `candidate_014`):	100/100, 122.46 s

Figures are sourced from the repository CSV exports and Git history.

Contents

1 Introduction and Objective **2**

2 Simple Heuristic **3**

2.1 Validation Rule 3

2.2 Search Loop 3

2.3 Evolution of `generate_initial_population` 3

2.4 Manual Score 5

3 FunSearch Architecture **7**

3.1 GPT-5.5 Candidate `candidate_006` 7

3.2 DeepSeek Candidate `candidate_014` 8

4 Experimental Results **8**

5 Scientific Discussion **8**

6 Dependencies and Libraries **9**

1 Introduction and Objective

GraphBench asks for automatic refutation of graph theory conjectures. A conjecture compares two graph invariants, for instance

$$y(G) \leq f(x(G)) \quad \text{or} \quad y(G) \geq f(x(G)).$$

A counterexample is a graph in the required benchmark class for which the inequality is strictly violated. The main benchmark classes are `connected`, `tree`, and `claw_free`.

The program must load conjectures, generate admissible graphs, compute the required invariants, maximize the violation, and output a graph in `graph6` format. Our solution has two parts. Part 1 is a hand-designed heuristic search. Part 2 implements a FunSearch-style loop in which a language model proposes scoring functions that are evaluated automatically.

The main result is that Part 1 refutes all 100 benchmark conjectures with a total cost of 25.280746 seconds, or 0.252807 seconds on average, on the final local v5 run. FunSearch also reaches 100/100 with some candidates, but it does not improve over the final hand-designed heuristic.

Part 1

2 Simple Heuristic

2.1 Validation Rule

Final validation uses only the mathematical violation. For a conjecture $y \leq f(x)$, we use

$$\text{violation}(G) = y(G) - f(x(G)).$$

For a conjecture $y \geq f(x)$, we use

$$\text{violation}(G) = f(x(G)) - y(G).$$

A graph is accepted only when the violation is strictly positive and the class constraints are verified after recomputing invariants. This separation matters: a heuristic score can guide search, but it never validates a false counterexample.

Module	Role
`loader.py`	Loads the benchmark CSV
`conjecture.py`	Conjecture object and exact violation formula
`invariants.py`	Invariant computation using `networkx` and `numpy`
`classes.py`	Tests for `connected`, `tree`, `claw_free`
`generators.py`	Initial graph families and random generation
`mutations.py`	Local graph modifications
`repair.py`	Repair after generation or mutation
`search.py`	Selection, mutation, evaluation, and stopping logic
`verify.py`	Strict verification of `graph6` counterexamples

Table 1: Part 1 architecture.

2.2 Search Loop

The main search keeps a scored population. It starts from an initial population, selects promising parents, applies a local mutation or injects a fresh graph, repairs the candidate if needed, and computes only the invariants required by the current conjecture. Candidates are sorted by objective score, but the stopping condition remains strict validation.

The evaluation path is intentionally robust to automatically generated scores. If a scorer raises an exception, returns `None`, or produces a non-finite value, its objective becomes $-\infty$ and the engine continues.

Listing 1: Candidate evaluation in the search engine.

```
def _evaluate(graph, conjecture, needed, scorer):
    invariants = compute_cached(graph, needed)
    try:
        raw_objective = scorer(graph, invariants, conjecture)
        objective = float(raw_objective) if raw_objective is not None else float("-inf")
    except Exception:
        objective = float("-inf")
    if not math.isfinite(objective):
        objective = float("-inf")
    return objective, violation_score(graph, invariants, conjecture), invariants
```

2.3 Evolution of `generate_initial_population`

The most decisive improvement in Part 1 was not score shaping alone, but changing the graph space exposed to the search engine. In v1, `generate_initial_population` sampled random graphs in the requested class. This explores broadly, but it misses rare extremal structures.

Listing 2: v1: repaired random population.

```
population = []
while len(population) < size and attempts < size * 80:
    n = rng.randint(min_order, max_order)
    graph = generate_graph(classes, rng, n)
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
```

In v2, we added a deterministic seed phase before random sampling. The population starts with simple graphs and targeted families. This changed the behavior of the system: instead of waiting for random mutations to discover the right shape, the engine begins close to useful counterexamples.

Listing 3: v2 and later: structural seeds before randomness.

```
for graph in seed_graphs(classes, min_order, max_order):
    graph = repair(graph, classes, rng)
    if graph is not None and satisfies_classes(graph, classes):
        population.append(_normalize_nodes(graph))
    if len(population) >= size:
        return population
```

The first seeds were paths, cycles, cliques, and spider trees. Spiders were added because they can increase domination-style invariants without increasing matching as quickly. They are built from a center and several legs of controlled lengths.

Listing 4: Spider seeds introduced in v2.

```
def _spider(lengths):
    graph = nx.Graph()
    graph.add_node(0)
    next_node = 1
    for length in lengths:
        previous = 0
        for _ in range(length):
            graph.add_edge(previous, next_node)
            previous = next_node
            next_node += 1
    return graph
```

In v3, the initial population was expanded with double-stars, damaged clique chains, sparse connected graphs, triangles with attached paths, and clique–path–clique graphs. Double-stars target cases with many leaves around few centers. Damaged clique chains keep high density and many triangles while changing maximum degree and bridge structure. Clique–path–clique graphs are useful when spectral quantities and distances react strongly to bottlenecks.

Listing 5: Example clique–path–clique seed.

```
def _clique_path_clique(left_size, right_size, path_length):
    left = list(range(left_size))
    path = list(range(left_size, left_size + path_length))
    right = list(range(left_size + path_length,
                       left_size + path_length + right_size))
    graph.add_edges_from((u, v) for i, u in enumerate(left)
                         for v in left[i + 1:])
    graph.add_edges_from((u, v) for i, u in enumerate(right)
                         for v in right[i + 1:])
    previous = left[0]
    for node in path:
        graph.add_edge(previous, node)
        previous = node
    graph.add_edge(previous, right[0])
```

v4 stabilized the last failures by adding branched paths and triangle tails. Branched paths remain trees but create strong local effects on independent domination, leaves, and Zagreb indices. Triangle tails preserve a near claw-free shape while controlling distance-related invariants.

v5 keeps the same graph families as v4, but makes the population size adaptive. The default size becomes 30 candidates, which reduces search overhead for most conjectures. It rises to 60, or exceptionally 120, for a few sensitive invariant pairs such as `second_zagreb_index`/`independent_domination_number`, `largest_distance_eigenvalue`/`proximity`, `average_degree`/`clique_number`, `clique_number`/`minimum_degree`, `radius`/`remoteness`, and `density`/`proximity`. This v5 keeps 100/100 valid counterexamples and reduces the measured total cost from 136.801705 seconds in v4 to 25.280746 seconds.

Version	Initial population additions	Motivation
v1	Random class-aware generation: trees, paths, cycles, cliques, `G(n,p)`, dense claw-free graphs.	Build a modular baseline and verify the pipeline.
v2	Deterministic seeds, fixed-size paths/cycles/cliques, spider trees.	Start near simple extremal graph families.
v3	Double-stars, damaged cliques, sparse graphs, triangle paths, clique–path–clique.	Target domination, density, triangles, distances, and spectral effects.
v4	Branched paths and triangle tails.	Resolve final tree, claw-free, and distance/spectral cases.
v5	Adaptive population size: 30 by default, 60 or 120 for a few sensitive pairs.	Speed up search while preserving strict 100/100 validation.

Table 2: Evolution of `generate_initial_population`.

2.4 Manual Score

The manual score starts from raw violation and adds small normalized guidance terms. The dominant coefficient remains the violation so that the search stays aligned with the mathematical objective.

Listing 6: Simplified manual score.

```
score = 100.0 * violation
score += 0.15 * y_value if conjecture.sign == "<=" else -0.15 * y_value
score += 0.12 * clamp(direction_from_derivative) * x_value
score += structural_guidance(G, invariants, conjecture)
```

The structural terms look at density, leaf ratio, triangles, and diameter. They encourage shapes

consistent with the invariant that should become large or small. In practice, these terms mostly broke ties between promising candidates; the main performance jump came from graph families.

Part 2

3 FunSearch Architecture

Part 2 implements a FunSearch-inspired loop. The model generates a Python scoring function named `heuristic_score`. It is stored in `src/graphbench/funsearch/candidates/` and evaluated by the same engine as Part 1. Only the score changes; generators, mutations, invariants, and verification are unchanged.

The cycle is:

1. read the registry of evaluated candidates;
2. select the best source files;
3. build a prompt with these sources and their results;
4. call OpenRouter for a new Python function;
5. extract and locally validate the function;
6. run the evaluator and append a row to `results/funsearch/registry.csv`.

We first used GPT-5.5 with low reasoning because it was expected to be strong at code and heuristic generation while staying cheaper than higher reasoning modes. For the longer 25-generation run, we switched to DeepSeek mainly for cost reasons.

Commit `7527406` corresponds to the first GPT-5.5 experiment. Its registry shows six main generations: `candidate_001` and `candidate_002` find 29/30 on a short evaluation, `candidate_003` finds 28/30, `candidate_004` finds 97/100, `candidate_005` finds 98/100, and `candidate_006` initially finds 95/100 in one registry row. After full reevaluation in `candidate_006-gpt5.5_100.csv`, `candidate_006` is the only candidate from that series to reach 100/100.

3.1 GPT-5.5 Candidate `candidate_006`

Candidate 006 remains close to the manual score, but it adds an interesting idea: reward graphs near the decision boundary. It multiplies violation by 50, then adds a Gaussian term that is largest when the violation is close to zero.

Listing 7: Excerpt from `candidate_006.py`.

```
score = 50.0 * violation
boundary_weight = math.exp(-5.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.0 * boundary_weight * diff_ratio
```

This is reasonable for local search: a graph that is almost a counterexample is often a better parent than a graph far away from the boundary. Candidate 006 also uses a polynomial derivative direction, mainly when violation is still negative.

Listing 8: Derivative guidance in `candidate_006.py`.

```
if sign == '<=':
    deriv_bonus = deriv / (abs(deriv) + 1.0)
else:
    deriv_bonus = -deriv / (abs(deriv) + 1.0)
if violation < 0:
    score += 0.5 * deriv_bonus
```

It also contains weak class-specific hints. For trees it rewards leaves; for claw-free graphs it mildly penalizes large maximum degree. These choices did not beat Part 1, but they produced a stable scorer compatible with the search engine.

3.2 DeepSeek Candidate `candidate_014`

In the 25-cycle DeepSeek sequence, `candidate_014` is the best registry candidate. It uses almost the same structure as 006, with a sharper boundary term and an explicit penalty when the derivative direction goes against the objective.

Listing 9: Excerpt from `candidate_014.py`.

```
boundary_weight = math.exp(-6.0 * violation * violation)
diff_ratio = diff_abs / (diff_abs + n + 1.0)
score += 3.5 * boundary_weight * diff_ratio

if sign == '<=':
    direction = deriv
else:
    direction = -deriv
dir_norm = direction / (abs(deriv) + 1.0)
away_penalty = max(0.0, -dir_norm) * 0.3
if violation < 0:
    score += 0.5 * dir_norm - away_penalty
```

Scientifically, this candidate is coherent: it keeps violation as the main signal, encourages boundary candidates, and avoids locally bad derivative directions. Its full v5 result is 100/100 with total cost 122.459858 seconds. It is valid and slightly faster than GPT-5.5 candidate 006, but still slower than the manual Part 1 v5 heuristic.

4 Experimental Results

Method	Found	Total cost	Average time
Part 1 hand-designed heuristic v5	100/100	25.28	0.253 s
Part 1 hand-designed heuristic v4	100/100	136.80	1.368 s
FunSearch DeepSeek candidate 014	100/100	122.46	1.225 s
FunSearch GPT-5.5 candidate 006	100/100	126.54	1.265 s

Table 3: Final results on the 100 conjectures.

Part 1 v5 is the fastest method on the final benchmark. It has median time 0.170065 seconds, maximum time 1.374284 seconds, and mean best score 2.877544. GPT-5.5 candidate 006 reaches 100/100 with total cost 126.543390 seconds. DeepSeek candidate 014 reaches 100/100 with total cost 122.459858 seconds and is the best candidate in the DeepSeek sequence.

The fact that FunSearch reaches 100/100 matters: the architecture generates executable and valid scoring functions. However, it does not improve final runtime. The most likely explanation is that the Part 1 engine was already strongly helped by its initial graph families. When the right graph is already present in the initial population or close to it, the fine details of the score become secondary.

5 Scientific Discussion

The easiest conjectures were those with simple extremal graph families: small complete graphs for clique or average-degree cases, paths for some distance cases, spider trees for total domination versus matching, and double-stars for independent domination and vertex cover.

The hardest conjectures often involved costly or sensitive invariants: distances, eigenvalues, domination, total domination, and independent domination. The spectral cases included the largest distance eigenvalue and the second-smallest Laplacian eigenvalue. Fixing `proximity` and `remoteness` was important because the benchmark uses reciprocal conventions:

$$\texttt{proximity} = 1/\max \overline{d}, \qquad \texttt{remoteness} = 1/\min \overline{d}.$$

Before that correction, several cases looked difficult for the wrong reason. More precisely, the most sensitive invariants include `largest_distance_eigenvalue`, `second_smallest_laplace_eigenvalue`, `domination_number`, `total_domination_number`, and `independent_domination_number`.

The main lesson is that graph-family generation mattered more than score shaping. **Local mutations** explore well around a good form, but they are weak at discovering rare structures from scratch. By adding the right seeds, we changed the geometry of the search space: the engine starts near regions where positive violations exist.

FunSearch reaches 100/100, which confirms that the architecture can generate valid executable scoring functions. However, in our final experiments it does not beat the best Part 1 hand-designed heuristic in total runtime.

AI was useful for proposing score functions, implementing the FunSearch loop, and exploring variants. It was less decisive for identifying extremal graph families and benchmark-specific invariant conventions. Human analysis was still necessary to verify invariants, interpret failures, choose seed families, and keep strict validation.

6 Dependencies and Libraries

The solution is deliberately lightweight. The `requirements.txt` file contains:

Dependency	Use
`networkx >= 3.2`	Graph representation, generators, connectivity, line graphs, graph6
`numpy >= 1.26`	Numerical computation, especially spectral invariants

Table 4: Direct Python dependencies.

The Python standard library is used for CSV files, paths, timing, dynamic imports of FunSearch candidates, reproducible randomness, and JSON handling. For Part 2, OpenRouter is called through a local client in the repository; the API key is not versioned. The report is written in LaTeX and compiled with `pdflatex`.